Kafka Integration with C#

Task 1 — Chat App using Kafka

Step 1 — Install NuGet Package

Create a C# Console Application and install Confluent.Kafka:

```
Install-Package Confluent.Kafka
```

Step 2 — Producer Code

Sends messages to the 'chat-message' Kafka topic.

```
using System; using
Confluent.Kafka;

class Producer
{
    static void Main(string[] args)
{
        var config = new ProducerConfig { BootstrapServers = "localhost:9092" };

        using (var producer = new ProducerBuilder<Null, string>(config).Build())
{             Console.WriteLine("Type message and press Enter. Ctrl+C to
exit.");             while (true)             {
                string message = Console.ReadLine();
producer.Produce("chat-message",
                    new Message<Null, string> { Value = message },
(dr) => {
                        if (dr.Error.Code != ErrorCode.NoError)
Console.WriteLine($"Error: {dr.Error.Reason}");                      else
Console.WriteLine($"Sent to: {dr.TopicPartitionOffset}");                 });
                producer.Flush(TimeSpan.FromSeconds(1));
}
        }
    }
}
```

Step 3 — Consumer Code

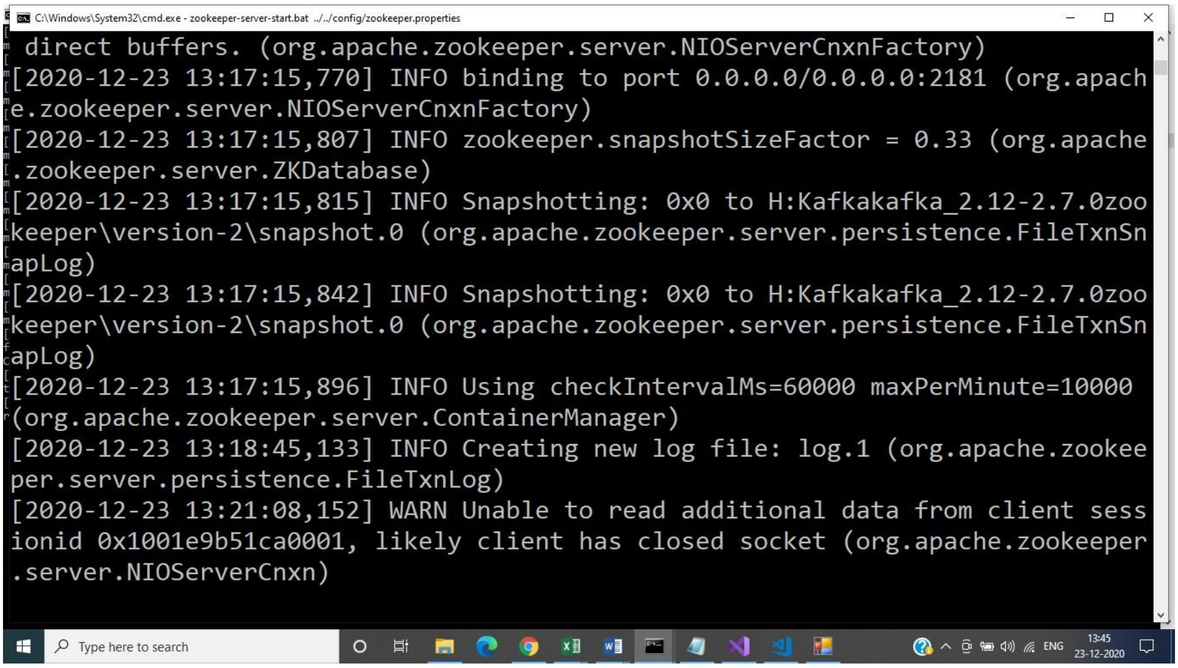
Consumes messages from the 'chat-message' topic in the command prompt.

```
using System; using
Confluent.Kafka;

class Consumer
{
    static void Main(string[] args)
    {
        var config = new ConsumerConfig
{
            GroupId          = "chat-group",
            BootstrapServers     =     "localhost:9092",
AutoOffsetReset   = AutoOffsetReset.Earliest         };

        using (var consumer = new ConsumerBuilder<Ignore, string>(config).Build())
{
            consumer.Subscribe("chat-message");
Console.WriteLine("Listening for messages...");
while (true)             {
                var cr = consumer.Consume();
Console.WriteLine($"Received: {cr.Message.Value}");
```

```
                }
            }
        }
    }
```

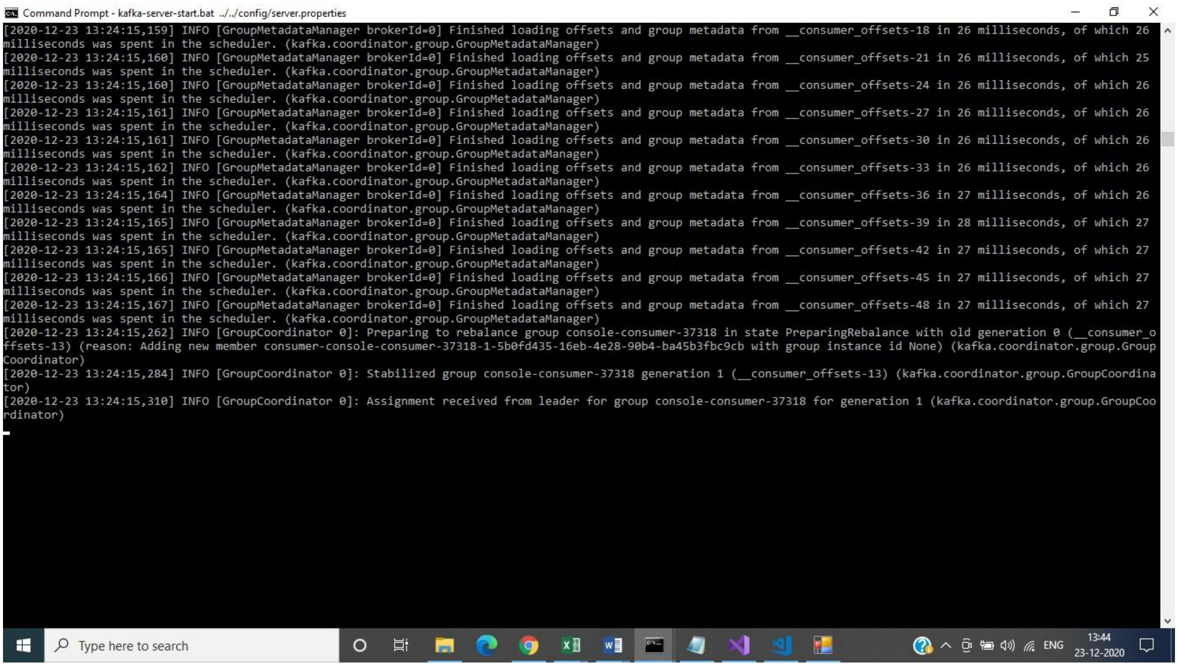
Step 4 — Start Zookeeper `zookeeper-server-start.bat ..\..\config\zookeeper.properties` Screenshot — Zookeeper

Running:



Step 5 — Start Kafka Server `kafka-server-start.bat ..\..\config\server.properties` Screenshot — Kafka

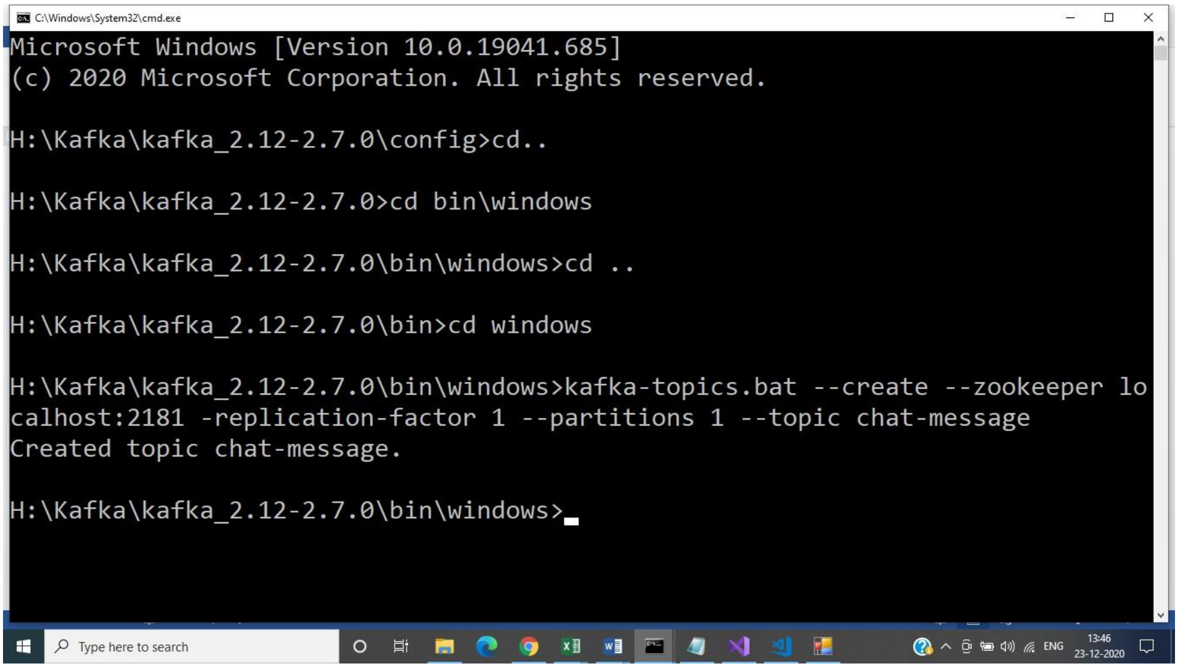
Server Running:



Step 6 — Create Topic

`kafka-topics.bat --create --zookeeper localhost:2181 --replication-factor 1 --partitions 1 --topic chat-message`

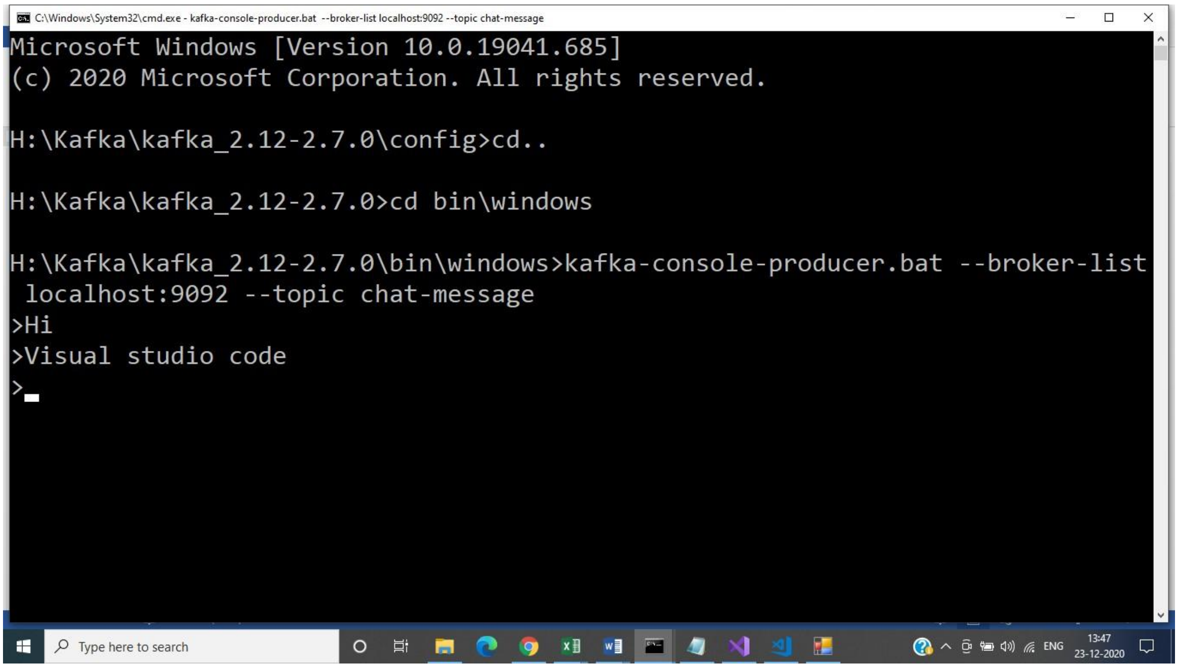
Screenshot — Topic Created:



Step 7 — Produce Messages

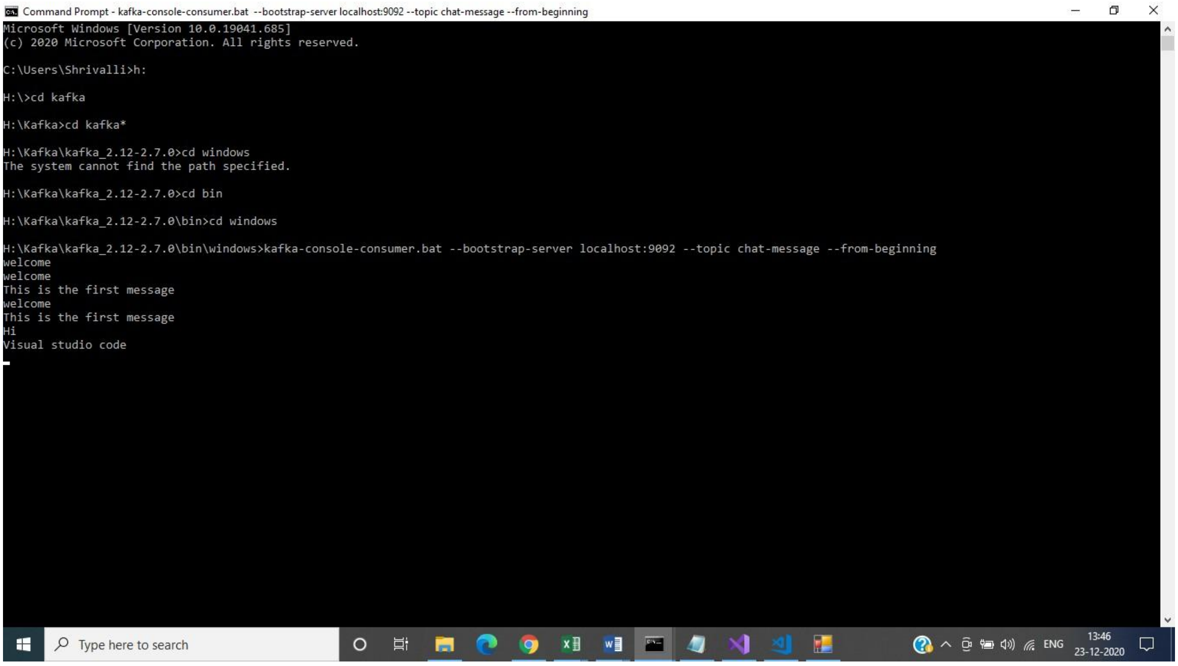
`kafka-console-producer.bat --broker-list localhost:9092 --topic chat-message`

Screenshot — Producer sending messages:



Step 8 — Consume Messages in Command Prompt

`kafka-console-consumer.bat --bootstrap-server localhost:9092 --topic chat-message --from-beginning` Screenshot — Consumer receiving messages:



Task 2 — Chat App using C# Windows Forms + Kafka

Create a chat application using a C# Windows Forms Application that uses Kafka to send messages and consume them in different client applications.

Step 1 — Create Windows Forms Project

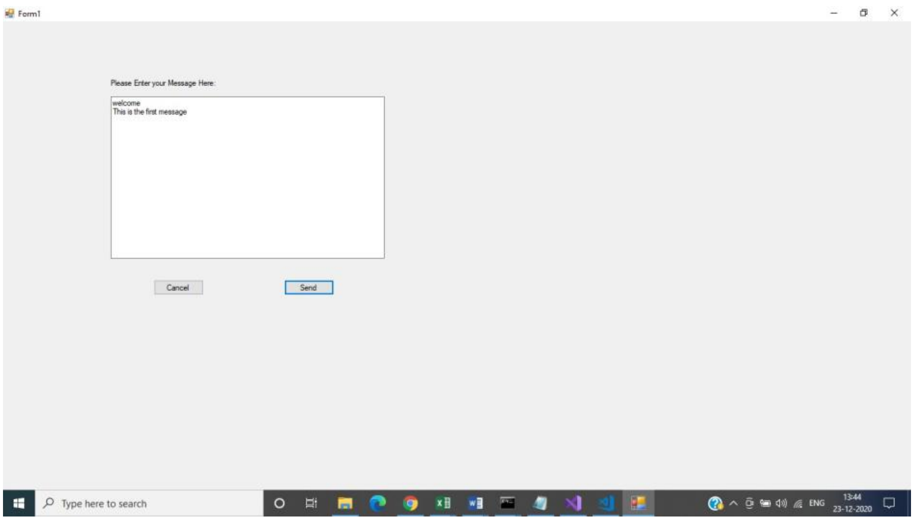
Create a new Windows Forms App (.NET Framework) in Visual Studio and install Confluent.Kafka:

```
Install-Package Confluent.Kafka
```

Step 2 — Form Design

```
Controls added to Form1:
  Label   : Text = "Please Enter your Message Here:"
  TextBox : Name = txtMessage, Multiline = True
  Button  : Name = btnSend,    Text = "Send"
  Button  : Name = btnCancel, Text = "Cancel"
```

Screenshot — Windows Form UI:



Step 3 — Form1.cs Code (Producer)

```
using System;
```

```
using System.Windows.Forms;
using Confluent.Kafka;

namespace KafkaChatApp
{
    public partial class Form1 : Form
{
        private IProducer<Null, string> _producer;

        public Form1()         {
InitializeComponent();
            var config = new ProducerConfig { BootstrapServers = "localhost:9092" };
_producer = new ProducerBuilder<Null, string>(config).Build();         }

        private async void btnSend_Click(object sender, EventArgs e)
{
            string msg = txtMessage.Text.Trim();
if (string.IsNullOrEmpty(msg)) return;

            await _producer.ProduceAsync("chat-message",
                new Message<Null, string> { Value = msg });

            MessageBox.Show("Message sent!", "Success",
MessageBoxButtons.OK, MessageBoxIcon.Information);
txtMessage.Clear();         }

        private void btnCancel_Click(object sender, EventArgs e)
{
            txtMessage.Clear();
}

        protected override void OnFormClosing(FormClosingEventArgs e)
{             _producer?.Dispose();
base.OnFormClosing(e);         }
    }
}
```

Step 4 — Consumer (separate app or Form)

The consumer can be the command-prompt consumer from Task 1, or a second Windows Forms app using a BackgroundWorker to poll messages:

```
// Form2 - Receiver (Consumer)
using System.ComponentModel;
using System.Windows.Forms;
using Confluent.Kafka;

public partial class Form2 : Form
{
    private BackgroundWorker _worker = new BackgroundWorker();

    public Form2()
{
        InitializeComponent();
        _worker.DoWork += Worker_DoWork;
_worker.RunWorkerAsync();     }

    private void Worker_DoWork(object sender, DoWorkEventArgs e)
{
        var config = new ConsumerConfig
{
            GroupId          = "winforms-group",
            BootstrapServers     =     "localhost:9092",
AutoOffsetReset  = AutoOffsetReset.Earliest         };
```

```
        using (var consumer = new ConsumerBuilder<Ignore, string>(config).Build())
{
            consumer.Subscribe("chat-message");
while (true)             {
                var cr = consumer.Consume();
Invoke(new Action(() =>
                    lstMessages.Items.Add(cr.Message.Value)));
}
        }
    }
}
```

Output — Messages received from the Windows Form

Messages typed and sent from the Windows Form appear in the consumer. Screenshot below shows the consumer receiving all messages: